



**UNIVERSITY
OF LONDON**

BSc Computer Science

CM2030 Graphics Programming

April 2026 session

Coursework 2: Image Processing Applications

Introduction

In this coursework, you will develop two interactive image processing applications using the provided images only. The tasks simulate real-world features found in modern media applications such as streaming platforms and smartphone camera software. Both tasks should be run from the same app. Use key interaction to switch between tasks: press 1 to load Task 1 and press 2 to load Task 2. Once a task is loaded, use the task-specific keys described below.

Task 1 – Streaming Carousel with Background Removal (40%)

Build an interactive image carousel similar to modern streaming platforms. See animation under **Task 1 demo** for guidance.

Requirements:

- Load and display at least 8 provided images in a horizontal carousel from the provided images.
- Enable interaction using keyboard i.e. key c to load the carousel, then key l (i.e. lower case L) to load the images, then key s to start the animation.
- Apply thresholding to remove the background in all the provided images. An RGB threshold (e.g. checking R, G, B) is not always sufficient for removing the unnecessary pixels. You must also use another colour space conversion approach. In the module, you have seen HSB, and you should use this as well to produce the best visual results. You have to try both colour space conversions for each image and see which one performs better. You must experiment with all 8 images, as threshold values may differ from one image to another. Store the threshold settings for each image in a 2D array in the form:

`thresholds[i] = [colourSpace, c1, c2, c3];`

where i is the image index, colourSpace = 0 for RGB and colourSpace = 1 for HSB. The remaining three values correspond to the threshold values for the chosen colour space - add any additional parameters as required for thresholding. This will allow your program to determine which colour space and thresholds to use when loading and processing each image. We expect the images to appear in the carousel without residual background pixels.

- Apply background removal by iterating through the threshold settings array and processing each image using its stored colour-space mode and threshold values
- Animate transitions (e.g., smooth scrolling, fade or zoom effects)
 - The processed foreground (i.e. people only) images should appear using the following animation sequence: (i) fade in, zoom in, fade out, then (ii) fade in, zoom out, fade out, then (i), (ii), and so on. Also, the person's image should move from left to right as seen in the demo.
 - The background image should also animate and smoothly move from left to right. You can use an image of your liking for the background.
 - Finally, the text should also use the same animation as the person but the text should move from right to left. You should not use `translate()` for these tasks.

Note: The embedded GIF is provided for guidance only and is intended to illustrate the required animation principles. You are not required to reproduce the exact same animation, but your solution should clearly demonstrate the required movement, transition, and presentation effects.

Task 2 – Panorama Motion Guide (40%)

Build a motion guidance system similar to the directional guide used in smartphone panorama mode. This task does not require live camera input and does not require full panorama stitching. Instead, you will work with the provided image pairs only. For each pair, your program should analyse how the object or scene has shifted between Frame A and Frame B, estimate the motion direction, and display that direction using a large arrow overlay. A successful solution will therefore simulate the guidance stage of panorama mode rather than the full image-capture or stitching process. See video guidance under **Task 2 demo**.

Requirements:

- Load the panorama screen (use p key)
- Load pairs of images (use i key)
- For each selected pair:
 - Frame A
 - Frame B

- Convert both images to grayscale (g key)
- Apply a simple edge filter to both grayscale images (e key)
- Produce an edge output image for each frame
- Apply thresholding to the edge output so that only the strongest visible outlines remain (t key)
- Allow the threshold value to be adjusted interactively using a slider if needed
- Use the thresholded edge image to compute the centroid (n key) for each frame by:
 - using all pixels that remain in the thresholded image
 - adding together all their x positions
 - adding together all their y positions
 - dividing each total by the number of selected pixels

Compare the centroid of Frame A (Cx1, Cy1) with the centroid of Frame B (Cx2, Cy2) by calculating:

$$dx = Cx2 - Cx1$$

$$dy = Cy2 - Cy1$$

Use dx and dy to determine whether the motion is left, right, up, down, or diagonal.

Your system must detect motion in the following directions:

LEFT, RIGHT, UP, DOWN, UP-LEFT, UP-RIGHT, DOWN-LEFT, DOWN-RIGHT

Display the detected direction using a large arrow overlay (d key)

Additional requirement

The provided image pairs demonstrate 4 directions only. You must create additional motion pairs from the provided images so that all remaining directions are also demonstrated. These additional pairs must be created manually by repositioning the provided images in GIMP or a similar application. Marks for motion estimation accuracy will be awarded across both the provided pairs and the additional generated pairs.

Extension (20%)

Since this is a creative module, we would like to give more marks for implementing further ideas. Also, you should write some words about it in the

commentary (see next paragraph). Please note that we will award marks for the uniqueness of your extension and how technically challenging it appears to have been. The extension is worth 20% of your mark. Extensions must demonstrate additional technical functionality (not only visual styling). You should create just one extension for either task 1 or 2. The extension is worth 20% of your mark.

Code quality

Indentation, spacing, OO structure, reusable functions, meaningful variable names, appropriately scoped variables, avoiding redundant code, sensible approach to commenting.

Commentary (≤ 500 words)

Included inside the main .js file.

Explain:

- Explain the app fully i.e. provide a walkthrough of the functionalities.
- What problems have you faced and were you able to solve them?
- Were you on target to successfully complete your project? If not, how would you address the issue/s and do things differently?
- Also discuss your extension and why it is a unique idea.

The report quality (i.e. language) will also be assessed. Be precise and deliver the information within 500 words. Include this in your main .js file.

General information

You should complete this work using the libraries shown in the module. Do not use external code for this assignment. All your code and commentary will be checked for plagiarism and AI generation. The use of AI tools, code generators, or externally sourced solutions is not permitted. Work submitted must be your own.

Submission requirements

1. Upload code as a .zip, including all libraries and any other files you may have used. *Test that your zip file can be downloaded, unzipped and that your app works fine without errors.*
2. Merge all your .js code in a single file, then upload it in the last prompt. Use the JavaScript Bundler Tool (see previous learning item) to merge all your .js files into a single .pdf file. Exclude any libraries you used such as p5.js, matter.js etc. **This is a submission requirement as we need this**

to run your code. You will receive zero points for the entire project if you fail to submit your code in this way.

Marking rubric [total: 50 marks]

Basic functionality (image loading and interaction) [5 marks]

Image processing (thresholding, background removal, edge detection) [16 marks]

Animation for task 1 [4 marks]

Motion estimation for task 2 [5 marks]

Code quality [7 marks]

Documentation [3 marks]

Creative extension [10 marks]

[END OF COURSEWORK ASSIGNMENT]